

STS3215 Serial Servo Bus (Kogna FPGA)

Support for up to 8 Feetech STS3215 (STS/SCS family) smart servos **daisy chained on a single wire** at 1 Mbaud (8-N-1, idle high, LSB first), carried by **IO0** (default) or **EX_IO_13** (selected by ServoBusEXIO, freeing IO0/IO1 for hardware encoder 0). Each servo has a unique ID (0–7) which is also its axis Input/Output Channel number.

The FPGA part (`sts_uart.vhd`) is deliberately dumb — a half-duplex byte-stream UART with a block-RAM packet buffer. **All protocol work lives in the DSP** (`DoSerialServos()` in `DSP_KOGNA\fast.c`): packet assembly, checksums, response validation, telemetry decoding, timeouts, and scheduling. This replaced an earlier design with six independent per-pin protocol engines (~2800 LUTs); the byte-stream design uses roughly a tenth of that, and one wire serves all servos because the servo's own ~80 ms response lag — not bus bandwidth — limits useful update rate.

Bus scheduling (done by the DSP)

Each bus cycle (~1 ms with all 8 servos, faster with fewer):

1. **SYNC_WRITE** (0x83) broadcasts acceleration/position/time/speed to every actively commanded servo in one packet — 8 + 8·N bytes, no replies.
2. **One telemetry READ** round robin: the 15-byte block 0x38–0x46 from one claimed servo (position, speed, load, voltage, temperature, status, moving, current) — 8 bytes out, 21 back, fully validated before use.

So position commands go out at ~1 kHz regardless of servo count, and each servo's telemetry refreshes every N cycles (~1 kHz / N). Aux requests (configuration reads/writes) slot between transactions.

The DSP spreads FPGA buffer transfers over interrupts (8-byte FPGA64 accesses, max 4 per interrupt) so the servo interrupt spends only a few µs here in the worst case.

FPGA register map (16-bit registers — FPGA64, buffers also via FPGA64)

Addr	R/W	Function
0x200	R/W	Bit 0 = bus enable; bit 1 = bus pin: 0 = IO0, 1 = EX_IO_13 (the I/O bit must also be defined as an output)
0x201	W	Packet length in bytes — writing starts transmission
0x201	R	Status: bit 15 TX busy, bit 14 RX overflow, bit 13 framing error, bits 8:0 received byte count
0x202	W	Bit 0 = clear receive buffer and error flags
0x240–0x27F	R/W	Transmit packet buffer, 64 words = 128 bytes, low byte first
0x280–0x2FF	R	Receive buffer, 128 words = 256 bytes, cleared at each transmission start

Transmit behavior: 2 bit-times of driven idle-high lead-in, the packet, 2 bit-times of driven guard after the final stop bit, then the pin releases for the servo's reply. Once the reply completes (and between transactions) the DSP engine PARKS the line — actively driven high as a GPIO output — so the bus never floats; no pull-up resistors are needed or fitted, and nothing fights the servos' own pull resistors as the chain grows. Received bytes are majority-vote sampled and stored whenever not transmitting.

Normally nothing accesses these registers directly — the DSP engine owns them whenever any axis uses a serial servo mode *or* an aux request is pending. Only a User C program doing raw bus experiments (with no axes configured for serial servo) should touch them.

DSP interface (the normal way to use all of this)

Axis integration is via the `SERIAL_SERVO_INPUT_MODE` / `SERIAL_SERVO_MODE` / `CL_SERIAL_SERVO_MODE` axis modes — see `SERIAL_SERVO_Axis_Mode.md`.

Telemetry arrives in DSP globals (see `KMotionDef.h`): `ServoPosition[]` (multi-turn accumulated), `ServoCurrent[]`, `ServoVolts[]`, `ServoLoad[]`, `ServoTemp[]`, `ServoStatus[]`, plus per-servo `ServoResult[]/ServoErrors[]/ServoStale[]/ServoPosValid[]` diagnostics. `ServoSpeed/ServoAccel` are sent to every servo in each `SYNC_WRITE`. `ServoBusEXIO` picks the bus pin (0 = IO0 default, 1 = EX_IO_13).

Configuration reads/writes of any servo control-table register go through the **aux mailbox** — fill in `ServoAuxID/Reg/Data/Two/Read`, set `ServoAuxGo=1`, and the engine performs it between cyclic transactions (result in `ServoAuxResult`, read data in `ServoAuxData`). Works whether or not any axis claims the bus (with no axes configured the engine runs just to service it).

Setting up a chain

1. Servos ship as ID 1. Connect them **one at a time** (the two connectors on each servo are parallel — daisy chaining is just cabling) and run `SerialServoSetID.c` with `TO_ID = 0, 1, 2...` as each is added. It also zeroes the return delay and verifies the servo answers on its new ID.
2. Wire the chain's signal to IO0 (or EX_IO_13 with `ServoBusEXIO=1`), common grounds, servo power from its own supply rail (~2.5 A stall each at 7.4 V, ~2.7 A at 12 V — a 6–8 servo chain wants a 20 A-class rail).
3. Configure axes (`SerialServoAxisInit.c`) — the DSP claims the bus, sets the pin direction, and telemetry starts flowing automatically.

Electrical notes (from the Feetech manual)

- Kogna IO pins are 3.3 V; the STS family's published high level can approach 5 V. Scope the DATA line before connecting directly and use a level-safe buffer if needed; a 220 Ω series resistor near the FPGA pin is good practice during development.
- Signal wire routed with its ground return, away from the motor-current wiring. Bulk capacitance near the servo power bus.
- Note IO0/IO1 are also hardware encoder 0's inputs — don't use encoder channel 0 while the bus is on IO0 (set `ServoBusEXIO=1` to move the bus to EX_IO_13 instead).

Dynamixel Protocol 2.0 support (XL430 etc.)

The same bus and engine also run ROBOTIS Dynamixel X-series servos — **one servo family on the bus at a time**, selected by the DSP global `ServoProtocol` (0 = Feetech STS default, 1 = Dynamixel Protocol 2.0). The FPGA is protocol-agnostic; the DSP engine switches packet formats (header FF FF FD 00, 16-bit lengths, CRC-16, byte stuffing, 16-bit register addresses with 1/2/4-byte fields — set `ServoAuxTwo` to 0/1/2).

Key differences from the Feetech servos:

- **Low lag:** with `Profile Velocity/Acceleration = 0` (the `Dx1Setup.c` default) the servo's internal trajectory generator is off and each Goal Position is acted on immediately — run axes with `CmdLead = 0` and no preview; measure with `SerialServoLagVsVel.c` before adding any.

- **Native multi-turn:** Extended Position mode (Operating Mode 4) reports signed 32-bit position over ± 256 turns — no unwrapping.
- **Torque must be explicitly enabled** (register 64) — goals are silently rejected with torque off. `DxlAxisInit.c` handles it.
- **EEPROM registers (address < 64) are only writable with torque off** — and the servo *reports* rejected writes in its status error byte (`ServoAuxErr`), unlike the Feetech ack.
- **No current sensor** — `ServoCurrent[ ]` reports Present Load (signed, 0.1 % units) for these servos.
- **Factory baud is 57600:** FPGA reg 0x200 bit 2 (DSP global `ServoBaud576`) drops the bus to 57600 so `DxlSetup.c` can configure a new servo up to 1 Mbaud in-system. Normal operation is always 1 Mbaud.
- **Voltage:** XL430 = 6.5–12 V (shares the STS3215 12 V rail). **The lookalike XL330 is 5 V class and is damaged above 6 V** — check the model printed on the servo before wiring power.

Measured XL430 characteristics and tuned recipe (2026-08-05, no load)

Lag vs velocity at the default servo P = 640: **constant ~47 ms time lag** from 750 counts/s up (85 ms at 250 due to a ~10-count stiction/deadband tail) — the classic velocity-error of the servo's own P loop, NOT firmware dead time. Unlike the STS3215 it **responds to gain**: $\text{lag} \approx 1/\text{loop-gain}$.

The tuned recipe (baked into `DxlAxisInit.c`):

- Servo P (reg 84) = **10000** → residual lag ~5.5 ms constant. **P is RAM and resets to 640 at servo power up** — the axis init must rewrite it.
- `CL_SERIAL_SERVO_MODE` with `KFLOP P/I/D = 0` and **FFVel = 0.0055** (seconds): a velocity feedforward advances the command by a constant *time*, exactly cancelling the servo's constant time lag. (This is why FFV works on the XL430 but failed on the STS3215, whose lag varies 4× with velocity.) `MaxOutput` must be nonzero or the FF is clamped away.
- `CmdDelay = 0`, `CmdLead = 0` — **no preview machinery at all**.

Result: **±4 counts (0.35°) at 1500 counts/s**, pure noise, no systematic tracking error — vs the STS3215's $\pm 7\text{--}8$ counts which required `CmdDelay=2500` plus the velocity-scheduled lead. Expect to re-tune P (and possibly the servo's D, reg 80) once real load inertia is on the shaft.

Utility / example programs

File	Purpose
<code>SerialServoSetID.c</code>	One-time: assign a servo's ID on the chain (<code>FROM_ID</code> → <code>TO_ID</code>), zero return delay, verify
<code>SerialServoAxisInit.c</code>	Configure axis 0 on servo ID 0: modes, safe enable, torque on
<code>SerialServoTuneLag.c</code>	Measure tracking lag, sweep servo P gain, set feed forward from the result
<code>SerialServoSweepLag.c</code>	Diagnostic sweeps of <code>ServoAccel</code> / <code>ServoSpeed</code> / cycle pacing vs lag
<code>SerialServoDiag.c</code>	Layered bring-up diagnostic: FPGA identity, raw PING, raw telemetry, engine test
<code>SerialServoScan.c</code>	Probe all 254 IDs and report which answer — finds a servo whose ID is unknown
<code>SerialServoConfig.c</code>	Dump and set the servo's internal parameters (PID, deadband, torque limits, protections, travel limits) with full per-register documentation

File	Purpose
SerialServoRecover.c	Raw-bus (engine-independent) diagnostic/repair of servo configuration: write tests with fault-flag/framing detail, optional factory restore (instruction 0x06) — servo must be alone on the bus for the restore
TestParamAddr.c	Verifies the TCC67 address-of-parameter fix (the root cause of the 2026-08 "stuck EEPROM register" saga) — every line must print PASS
DxlSetup.c	One-time Dynamixel setup: find (1 M or factory 57600 baud), set 1 Mbaud, ID, return delay 0, Extended Position mode, profiles 0/0 (low lag)
DxlAxisInit.c	Configure axis 0 on a Dynamixel: modes, torque enable, telemetry wait, enable — low-lag (no CmdLead)
DxlConfig.c	Dump and set Dynamixel parameters (PID, feedforwards, profiles, limits, homing offset) with write rejection reporting
SerialServoLagVsVel.c	Measure tracking lag vs velocity — the table behind the lead schedule
SerialServoLeadSchedule.c	Velocity-scheduled CmdLead thread (the preview compensation runtime)

Measured characteristics (12 V C018)

- Tracking lag $\approx$ **80 ms**, dominated by servo-internal dead time. Servo P gain (reg 0x15) barely moves it; **feed forward is the fix** (FFVel $\approx$ lag).
- ServoAccel = 0 means "no limit" (identical to max 254). **Never use a small nonzero value** — 10 doubles the lag.
- Servo speed cap doesn't affect tracking down to $\sim$ 1200 counts/s.
- The command pipeline (FPGA + DSP) contributes well under 1 ms.
- The 2026-08 "corrupted EEPROM cell" saga was a compiler bug, not the servo.** Writes of P (reg 0x15) through the aux mailbox kept "reading back" 254/0 — the servo was faithfully storing garbage values that SerialServoConfig.c transmitted, because TCC67 miscompiled the **address of a function parameter** (`Wr(int val)` passed `&val`; C67 register-passed parameters are homed to stack slots, but the address-of path didn't translate to them). Fixed in TCC67 2026-08-03 (`i386-gen.c`, `load()` VT_LOCAL address branch); `TestParamAddr.c` verifies the fix on hardware. Older TCC67 binaries in the field still have the bug — in example code, never pass the address of a parameter; copy it to a local first.
- EEPROM write commits can stall the servo's ack tens of ms** (measured >21 ms of ack silence on the raw bus). The DSP engine waits up to +200 ms for the ack of any aux write to an EEPROM register (below 0x28) so a late ack is consumed instead of colliding with the next packet, then inserts $\sim$ 200 ms of bus silence; `SerialServoConfig.c` additionally pauses the cyclic traffic (large `ServoCycleTicks`), waits 250 ms before each verify read, and retries on mismatch.
- A write ack proves receipt, not application.** The Feetech status packet's error byte carries hardware fault flags only (voltage/sensor/ temp/current/angle/overload) — there is no write-rejected or range-error bit, and a locked EEPROM silently accepts-but-doesn't-persist. Readback is the only verification. The engine exposes the ack's fault flags as `ServoAuxErr` (-1 = no ack seen).
- Recovering a corrupted register:** `SerialServoRecover.c` (raw bus) diagnoses and repairs. Factory restore is instruction **0x06** — current Feetech SDKs name it `INST_RECOVERY`; do **not** use 0x0A (their `INST_RESET`), which only clears the multi-turn counter. Factory baud is 1 Mbaud (bus stays alive) but the ID may revert to factory 1, so the servo must be **alone on the bus** for a restore. Beyond that:

Feetech's Windows FD software (Programming tab: load a healthy servo's saved parameter set; Upgrade tab: firmware re-flash), or replace (~\$15).